DAILY DOUGH: SYSTEM OVERVIEW & DATABASE STRUCTURE

An On-Demand E-Commerce Platform for Donut Ordering and Delivery

Course: ITE 7 – Application Development & Emerging Technologies

Platform: Flutter (Android) & Web Admin Dashboard

Backend Database: Google Firebase Realtime Database (RTDB)

Documentation Release: Version 2.0 (Official Capstone & Defense Guide)

---

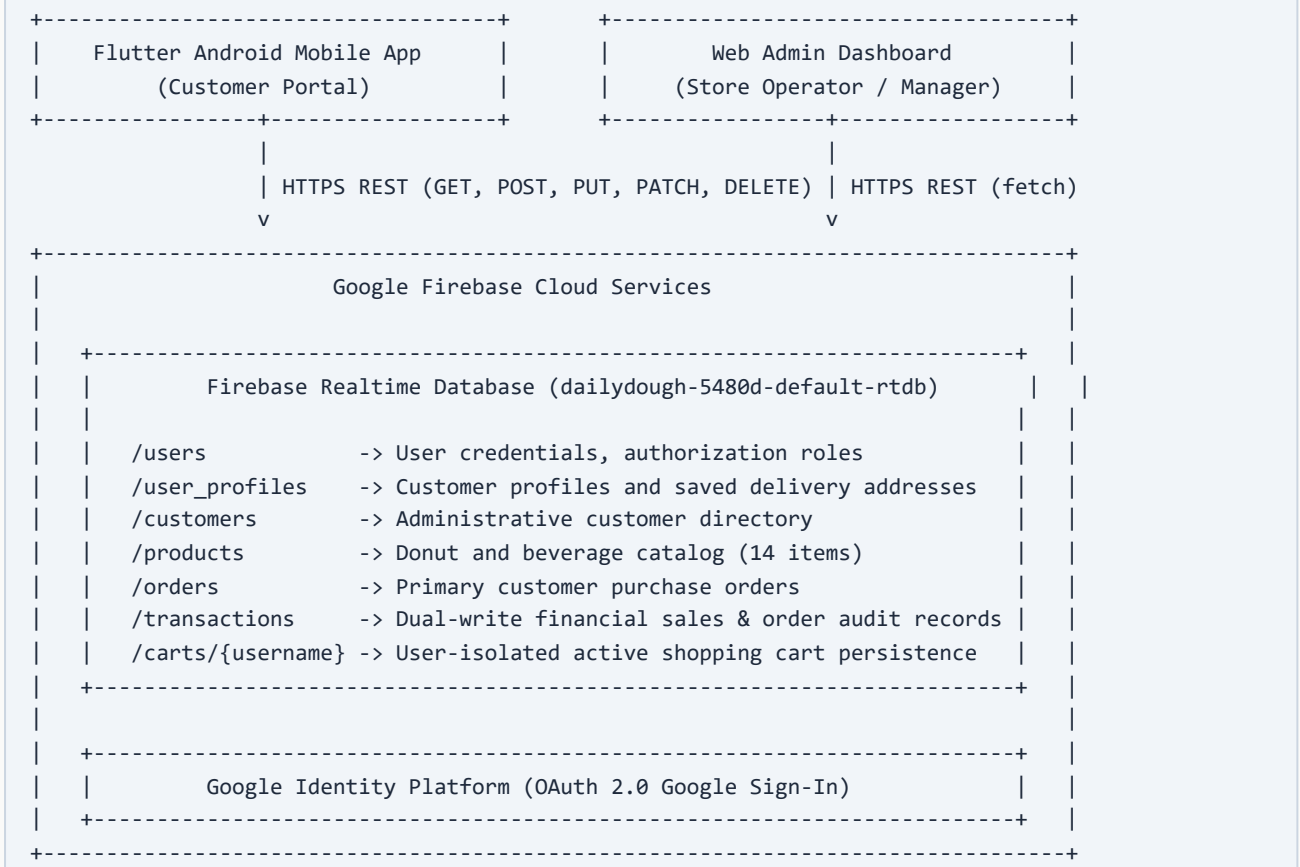
1. Executive Summary & Project Description

Daily Dough is an integrated mobile and web e-commerce solution engineered to digitize the ordering, culinary preparation, delivery tracking, and administrative orchestration of fresh handcrafted donuts, specialty pastries, and premium brewed coffee beverages. The platform pairs a high-performance native Flutter Android client with a lightweight responsive Web Admin Dashboard, powered by Google Firebase Realtime Database via direct REST APIs over HTTPS.

2. System Architecture & Component Design

The system follows a **Serverless Client-Cloud Decoupled Architecture**. Both client applications communicate directly with Google Firebase services without requiring an intermediate custom server.

SYSTEM ARCHITECTURE OVERVIEW



3. Technology Stack Summary

Category	Technology	Implementation in Daily Dough
Mobile Frontend	Flutter SDK / Dart 3.x	Native Android mobile app (`mobile_app/`) with 4-tab bottom navigation, clean treats grid, and Hero animations.
Web Frontend	HTML5, CSS3, Vanilla JS	Responsive single-page Web Admin Dashboard (`web_dashboard/`) with KPI metrics and live order controls.
Cloud Database	Firebase Realtime Database	Cloud NoSQL flat JSON tree storing credentials, catalogs, carts, orders, and sales records.
Authentication	Firebase Auth & Google OAuth 2.0	Dual authentication supporting email/password accounts and one-tap Google Sign-In.
State Management	Flutter Provider / ChangeNotifier	Reactive state synchronization across cart calculations and database polling listeners.
Build Tooling	Gradle 9.3.1 / Android SDK	Builds optimized release APK (`app-release.apk`, ~57.1 MB) with official Daily Dough branding.

4. Firebase Realtime Database Structure

Live analysis of the Firebase root at `https://dailydough-5480d-default-rtdb.firebaseio.com/` confirms 7 distinct data nodes:

FIREBASE REALTIME DATABASE JSON TREE HIERARCHY

```
dailydough-5480d-default-rtdb/
├── users/
│   └── <userId_or_pushKey>/
│       ├── authProvider : "password" | "google"
│       ├── email        : "customer@example.com"
│       ├── fullName     : "John Doe"
│       ├── password     : "[PROTECTED]"
│       ├── role         : "customer" | "admin"
│       └── username     : "johndoe"
├── user_profiles/
│   └── <sanitized_username>/
│       ├── fullName  : "John Doe"
│       ├── email     : "customer@example.com"
│       ├── phone     : "09171234567"
│       └── addresses : [{"address": "123 Main St", "label": "Home", "isDefault": true}]
├── customers/
│   └── <customerPushId>/
│       ├── fullName, email, phone, username, createdAt, accountType
├── products/
│   ├── treat_pistachio/
│   │   ├── name : "Pistachio", price: 49.0, stock: 50, imageUrl: "...", details: "..."
│   └── <productId>/ ... (14 treats & coffee items)
├── orders/
│   └── <orderId_or_pushKey>/
│       ├── orderId, customerName, username, userId, totalAmount, paymentMethod, status, timestamp
│       └── items : [{"name": "Pistachio", "price": 49.0, "quantity": 2}]
├── transactions/
│   └── <sharedKey>/ (Identical mirrored structure to /orders for sales auditing)
└── carts/
    └── <username>/
        └── <cartItemPushId>/
            ├── productName : "Pistachio"
            └── price       : 49.0
```

5. Verified Implementation Status Matrix

Feature / Component	Actual Status	Source Evidence	Functional Notes
Customer Registration	Completed	`register_screen.dart`	Validates input rules; writes records to `/users` and `/customers`.
Customer Login	Completed	`login_screen.dart`	Credential authentication with dynamic password visibility toggle.
Google Sign-In	Completed	`login_screen.dart`	One-tap OAuth 2.0 authentication using Google Play Services.
Treats Tab Clean Browsing	Completed	`products_screen.dart`	Clean 2-column grid showing only product image and name without inline cart clutter.
Product Detail Interface	Completed	`product_detail_screen.dart`	Hero image transition, pricing, culinary description, quantity controls (-/+), Add to Cart.
Shopping Cart	Completed	`cart_screen.dart`	Groups items by name, calculates subtotal and delivery fee, clear cart safeguard.
Checkout & Payment	Completed	`checkout_screen.dart`	Address selection, Cash on Delivery, and GCash reference number submission.
5-Stage Order Tracking	Completed	`order_tracking_screen.dart`	Real-time progress tracker: Placed -> Preparing -> Ready -> Out for Delivery -> Delivered.
Admin KPI Dashboard	Completed	`web_dashboard/app.js`	Computes live Total Sales, Orders Count, Active Customers, and Catalog Count.
Admin Orders Management	Completed	`web_dashboard/app.js`	Dual-node PATCH to `/orders` and `/transactions` ensuring 100% status persistence.
Admin Catalog Management	Completed	`web_dashboard/admin.html`	Add New Product modal, inline price/stock editor, deletion safeguard.
Admin Customer Directory	Completed	`web_dashboard/admin.html`	Complete list of registered customers, contact info, and lifetime order counts.

6. Academic Analysis: Cloud Firestore vs. Realtime Database

> **Key Defense Talking Point for ITE 7 Curriculum**

> The ITE 7 academic module references Cloud Firestore collections and documents. However, Daily Dough was engineered using Firebase Realtime Database (RTDB). Both products are official Google Firebase NoSQL solutions. RTDB was selected because its flat JSON tree structure provides sub-second reactive updates for active cart calculations and on-demand order fulfillment tracking, while minimizing client APK size (~57.1 MB) by avoiding heavy native client SDK dependencies.

7. Conclusion

Daily Dough represents a fully realized, defensible, production-grade e-commerce application. By seamlessly uniting a Flutter mobile client and an HTML5/JavaScript administrative portal through Google Firebase Realtime Database, it exemplifies best practices in reactive state management, asynchronous REST communication, and transactional data integrity.